

```

import { useState, useEffect, useRef } from "react";

const TABS = ["ホーム", "依頼", "地図", "業者", "タイムライン"];
const AV_COLORS =
["#f90,#e74c3c","#3498db,#2c3e50","#2ecc40,#27ae60","#9b59b6,#8e44ad","#e91e63,#c2185b",
"#1abc9c,#16a085","#f1c40f,#f39c12","#e74c3c,#c0392b"];
const getAvColor = n => { let s=0; for(let i=0;i<n.length;i++)s+=n.charCodeAt(i); return
AV_COLORS[s%AV_COLORS.length]; };
const getAvChar = n => n?n[0]:' ? ';
const timeAgo = ts => {
  const s=Math.floor((Date.now()-ts)/1000);
  if(s<60)return `${s}秒前`;
  if(s<3600)return `${Math.floor(s/60)}分前`;
  if(s<86400)return `${Math.floor(s/3600)}時間前`;
  return `${Math.floor(s/86400)}日前`;
};

function Avatar({ name, size=42 }) {
  const c = getAvColor(name||'?');
  return (
    <div
      style={{width:size,height:size,borderRadius:'50%',background:`linear-gradient(135deg,$${c})`,
        display:'flex',alignItems:'center',justifyContent:'center',fontSize:size*0.42,fontWeight:700,color:'fff',flexShrink:0}}>
      {getAvChar(name||'?')}
    </div>
  );
}

function calcDistanceKm(la1,lo1,la2,lo2){const
R=6371,dL=(la2-la1)*Math.PI/180,dO=(lo2-lo1)*Math.PI/180,a=Math.sin(dL/2)**2+Math.cos(la1*Math.PI/180)*Math.cos(la2*Math.PI/180)*Math.sin(dO/2)**2;return
R*2*Math.atan2(Math.sqrt(a),Math.sqrt(1-a));}
async function geocode(q){const r=await
fetch(`https://nominatim.openstreetmap.org/search?q=${encodeURIComponent(q)}&format=json&limit=1`,{headers:{"Accept-Language":"ja"}});const d=await r.json();if(!d.length)throw
new Error("not found");return{lat:parseFloat(d[0].lat),lon:parseFloat(d[0].lon)};}

function LeafletMap({currentPos,fromPos,toPos}){
  const ref=useRef(null),mapRef=useRef(null),markersRef=useRef([]),lineRef=useRef(null);
  useEffect(()=>{
    if(!document.getElementById("lf-css")){const
l=document.createElement("link");l.id="lf-css";l.rel="stylesheet";l.href="https://cdnjs.cloudflare.com/ajax/libs/leaflet/1.9.4/leaflet.min.css";document.head.appendChild(l);}
    const boot=()=>=>{if(mapRef.current)return;const
L=window.L,m=L.map(ref.current,{center:[35.68,139.77],zoom:5});L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png",{maxZoom:19,attribution:"© OSM"}).addTo(m);mapRef.current=m;draw();};

```

```

    if(window.L)boot();
    else{if(!document.getElementById("lf-js")){const
s=document.createElement("script");s.id="lf-js";s.src="https://cdnjs.cloudflare.com/ajax/libs/le
afler/1.9.4/leaflet.min.js";s.onload=boot;document.head.appendChild(s);}else{const
w=setInterval(()=>{if(window.L){clearInterval(w);boot();}},100);}}
    return()=>{if(mapRef.current){mapRef.current.remove();mapRef.current=null;}};
    },[]);
    function draw(){const
L=window.L,m=mapRef.current;if(!L||!m)return;markersRef.current.forEach(x=>m.removeLay
er(x));markersRef.current=[];if(lineRef.current){m.removeLayer(lineRef.current);lineRef.curre
nt=null;}
    const ic=c=>L.divIcon({html:`<div
style="background:${c};width:13px;height:13px;border-radius:50%;border:3px solid
#fff;box-shadow:0 0 6px ${c}"></div>`,iconSize:[13,13],iconAnchor:[6,6],className:""});

    if(currentPos)markersRef.current.push(L.marker([currentPos.lat,currentPos.lon],{icon:ic("#34
98db"))}).addTo(m).bindPopup("📍 現在地");

    if(fromPos)markersRef.current.push(L.marker([fromPos.lat,fromPos.lon],{icon:ic("#2ecc40"))}
).addTo(m).bindPopup("🟢 出発地");

    if(toPos)markersRef.current.push(L.marker([toPos.lat,toPos.lon],{icon:ic("#e74c3c"))}).addTo(
m).bindPopup("🔴 目的地");

    if(fromPos&&toPos){lineRef.current=L.polyline([[fromPos.lat,fromPos.lon],[toPos.lat,toPos.lo
n]],{color:"#f90",weight:3,dashArray:"8
6"}).addTo(m);m.fitBounds([[fromPos.lat,fromPos.lon],[toPos.lat,toPos.lon]],{padding:[50,50]
});}
    else if(currentPos)m.setView([currentPos.lat,currentPos.lon],14);}
    useEffect(()=>{if(mapRef.current&&window.L)draw();},[currentPos,fromPos,toPos]);
    return <div ref={ref} style={{width:"100%",height:260,borderRadius:12,border:"1px solid
#333",overflow:"hidden"}}/>;
  }

  const initialPosts = [
    {id:1,username:"代行太郎",text:"今日も元気に営業中！渋谷・新宿エリア対応できます
🚗",time:Date.now()-3600000,likes:5,liked:false,replies:[{id:101,username:"鈴木",text:"いつも
お世話になってます！",time:Date.now()-3000000}]},
    {id:2,username:"夜の代行",text:"深夜の代行依頼お待ちしております。港区・品川区エリア専門で
す🌙",time:Date.now()-7200000,likes:3,liked:false,replies:[]},
    {id:3,username:"ユーザーA",text:"先日お願いした代行さん、すごく丁寧でした！おすすめです
👍",time:Date.now()-86400000,likes:12,liked:false,replies:[{id:301,username:"代行太郎
",text:"ありがとうございます！またよろしくお願いします😊",time:Date.now()-80000000}]},
  ];

  function Tweet({post, myName, onLike, onReply}){
    const [replyOpen, setReplyOpen] = useState(post.replies.length>0);
    const [replyText, setReplyText] = useState("");

```

```

const [now, setNow] = useState(Date.now());
useEffect(()=>{const
t=setInterval(()=>setNow(Date.now()),30000);return(()=>clearInterval(t);,[]);

return (
  <div style={{display:"flex",gap:10,padding:"14px 16px",borderBottom:"1px solid
#1e1e1e"}}>
    <Avatar name={post.username} size={42}/>
    <div style={{flex:1,minWidth:0}}>
      <div
style={{display:"flex",alignItems:"baseline",gap:6,flexWrap:"wrap",marginBottom:4}}>
        <span style={{fontWeight:700,fontSize:15,color:"#fff"}}>{post.username}</span>
        <span
style={{color:"#555",fontSize:13}}>@{post.username.toLowerCase().replace(/s/g,"_")}</span
>
        <span
style={{color:"#555",fontSize:12,marginLeft:"auto",whiteSpace:"nowrap"}}>{timeAgo(post.tim
e)}</span>
      </div>
      <div
style={{fontSize:15,color:"#e8e8e8",lineHeight:1.6,wordBreak:"break-word",marginBottom:10
}}>
        {post.text.split("\n").map((l,i)=><span key={i}>{l}<br/></span>)}
      </div>
      {/* アクション */}
      <div style={{display:"flex",gap:0}}>
        <button onClick={()=>setReplyOpen(!replyOpen)}
style={{display:"flex",alignItems:"center",gap:5,background:"none",border:"none",cursor:"poin
ter",color:replyOpen?"#3498db":"#555",fontSize:13,padding:"6px 12px 6px
0",fontFamily:"inherit"}}>
          <span style={{fontSize:16}}>💬</span>
          <span>{post.replies.length||""}</span>
        </button>
        <button onClick={()=>onLike(post.id)}
style={{display:"flex",alignItems:"center",gap:5,background:"none",border:"none",cursor:"poin
ter",color:post.liked?"#e74c3c":"#555",fontSize:13,padding:"6px 12px 6px
0",fontFamily:"inherit"}}>
          <span style={{fontSize:16}}>{post.liked?"❤️":"💜"}</span>
          <span>{post.likes||""}</span>
        </button>
      </div>
      {/* リプライ欄 */}
      {replyOpen && (
        <div style={{marginTop:10,background:"#0a0a0a",border:"1px solid
#222",borderRadius:12,padding:"10px 12px"}}>
          {post.replies.map(r=>(
            <div key={r.id} style={{display:"flex",gap:8,padding:"6px 0",borderBottom:"1px solid
#1a1a1a"}}>

```

```

    <Avatar name={r.username} size={28}/>
    <div style={{flex:1}}>
      <div style={{fontWeight:700,fontSize:13,color:"#ccc"}}>{r.username}</div>
      <div
style={{fontSize:13,color:"#aaa",marginTop:2,lineHeight:1.4,wordBreak:"break-word"}}>{r.text
}</div>
      <div style={{fontSize:11,color:"#444",marginTop:2}}>{timeAgo(r.time)}</div>
    </div>
  </div>
))}
<div style={{display:"flex",gap:8,alignItems:"center",marginTop:8}}>
  <input value={replyText} onChange={e=>setReplyText(e.target.value)}

onKeyDown={e=>{if(e.key==="Enter"&&!e.shiftKey){e.preventDefault();if(replyText.trim()){on
Reply(post.id,replyText,myName);setReplyText("");}}}
    placeholder="返信する..." maxLength={100}
    style={{flex:1,background:"#111",border:"1px solid
#333",borderRadius:20,padding:"8px
14px",color:"#fff",fontSize:13,outline:"none",fontFamily:"inherit"}}/>
    <button
onClick={()=>{if(replyText.trim()){onReply(post.id,replyText,myName);setReplyText("");}}}
    style={{padding:"8px 14px",background:"#1a6b3a",color:"#4efa8a",border:"1px
solid
#2ecc70",borderRadius:20,fontSize:13,fontWeight:700,cursor:"pointer",fontFamily:"inherit",w
hiteSpace:"nowrap"}}>
      返信
    </button>
  </div>
</div>
)}
</div>
</div>
);
}

```

```

export default function App(){
  const [tab,setTab]=useState("ホーム");
  const [myStatus,setMyStatus]=useState("空き");
  const [agents,setAgents]=useState([
    {id:1,name:"代行A",phone:"090-1234-5678",status:"空き",area:"東京都渋谷区",rating:4.8},
    {id:2,name:"代行B",phone:"080-9876-5432",status:"対応中",area:"東京都新宿区",rating:4.5},
    {id:3,name:"代行C",phone:"070-1111-2222",status:"空き",area:"東京都港区",rating:4.9},
  ]);
  const [newAgent,setNewAgent]=useState({name:"",phone:""});
  const [requests,setRequests]=useState([
    {id:1,from:"田中",area:"渋谷区",time:"23:00",status:"待機中"},
    {id:2,from:"鈴木",area:"新宿区",time:"22:30",status:"対応中"},
  ]);
}

```

```

]);
const [newReq,setNewReq]=useState({from:"",area:"",time:""});
const [posts,setPosts]=useState(initialPosts);
const [myName,setMyName]=useState("");
const [postText,setPostText]=useState("");
const [emailSent,setEmailSent]=useState(false);
const [notifPerm,setNotifPerm]=useState("default");
const [currentPos,setCurrentPos]=useState(null);
const [fromPos,setFromPos]=useState(null);
const [toPos,setToPos]=useState(null);
const [fromInput,setFromInput]=useState("");
const [toInput,setToInput]=useState("");
const [geoMsg,setGeoMsg]=useState("");
const [calcMsg,setCalcMsg]=useState("");
const [distance,setDistance]=useState(null);
const nextId=useRef(200);

```

```

useEffect(()=>{"Notification" in window&&setNotifPerm(Notification.permission);},[]);

```

```

const requestNotif=async()=>{if(!("Notification" in
window)||Notification.permission!=="default")return;const r=await
Notification.requestPermission();setNotifPerm(r);};
const sendNotif=(title,body)=>{if("Notification" in
window&&Notification.permission==="granted")new
Notification(title,{body,icon:"https://cdn-icons-png.flaticon.com/512/684/684908.png"});};

```

```

const
notifStyle=notifPerm==="granted"?{bg:"#1a3a1a",color:"#2ecc40",border:"#2ecc40",text:"🔔
通知ON・依頼が来たら通知します
"}:notifPerm==="denied"?{bg:"#3a1a1a",color:"#e74c3c",border:"#e74c3c",text:"🔔 通知がブ
ロックされています"}:{bg:"#1a2a3a",color:"#6af",border:"#3498db",text:"🔔 通知を許可する
(依頼が来たらお知らせします)"};

```

```

const
addAgent=(()=>{if(!newAgent.name.trim())return;setAgents([...agents,{id:Date.now(),...newAg
ent,status:"空き",area:"未設定",rating:5.0}]);setNewAgent({name:"",phone:""});});
const addRequest=(()=>{if(!newReq.from.trim())return;const
r={id:Date.now(),...newReq,status:"待機中
"};setRequests([r,...requests]);setNewReq({from:"",area:"",time:""});sendNotif("🚗 新しい依頼
が届きました！","`${r.from}さん・${r.area}・${r.time}`");});

```

```

const addPost=(()=>{if(!postText.trim())return;const name=myName.trim()||"匿名ユーザー
";setPosts([{id:nextId.current++,username:name,text:postText,time:Date.now(),likes:0,liked:fa
lse,replies:[],...posts}]);setPostText("");});
const
handleLike=id=>setPosts(posts.map(p=>p.id===id?{...p,liked:!p.liked,likes:p.liked?p.likes-1:p
.likes+1}:p));

```

```

const
handleReply=(id,text,name)=>setPosts(posts.map(p=>p.id===id?{...p,replies:[...p.replies,{id:
nextId,current++,username:name||"匿名ユーザー",text,time>Date.now()}]:p));

const getLocation=()=>{if(!navigator.geolocation){setGeoMsg("❌ 位置情報が使えません");return;}setGeoMsg("取得中...");navigator.geolocation.getCurrentPosition(p=>{const pos={lat:p.coords.latitude,lon:p.coords.longitude};setCurrentPos(pos);setFromPos(pos);setFromInput("現在地");setGeoMsg("✅ 現在地を取得しました");},()=>setGeoMsg("❌ 取得失敗(許可を確認してください)"));};

const doCalc=async()=>{if(!fromInput.trim()||!toInput.trim()){setCalcMsg("⚠️ 両方入力してください");return;}setCalcMsg("計算中...");setDistance(null);setToPos(null);try{const fPos=(fromInput==="現在地"&&fromPos)?fromPos:await geocode(fromInput);const tPos=await geocode(toInput);setFromPos(fPos);setToPos(tPos);const d=calcDistanceKm(fPos.lat,fPos.lon,tPos.lat,tPos.lon);setDistance(d);setCalcMsg("✅ 計算完了");}catch{setCalcMsg("❌ 場所が見つかりませんでした");}};

const openGmap=()=>{if(!fromInput.trim()||!toInput.trim()){setCalcMsg("⚠️ 両方入力してください");return;}window.open(`https://www.google.com/maps/dir/?api=1&origin=${encodeURIComponent(fromInput)}&destination=${encodeURIComponent(toInput)}&travelmode=driving`,`_blank`);};

const S={
  input:{width:"100%",background:"#111",border:"1px solid #333",borderRadius:8,padding:"10px 12px",color:"#fff",fontSize:14,outline:"none",fontFamily:"inherit",boxSizing:"border-box"},
  card:{background:"#1a1a1a",borderRadius:12,padding:14,marginBottom:10},
  outlineBtn:{width:"100%",marginTop:10,padding:"11px 0",background:"none",border:"1px solid #f90",borderRadius:8,color:"#f90",fontWeight:700,fontSize:14,cursor:"pointer"},
};

return (
  <div style={{fontFamily:"Hiragino Sans','Noto Sans JP',sans-serif,background:"#111",minHeight:"100vh",color:"#fff",maxWidth:480,margin:"0 auto",display:"flex",flexDirection:"column"}}>

    { /* Header */ }
    <div style={{background:"#000",padding:"14px 16px 0",borderBottom:"1px solid #222",position:"sticky",top:0,zIndex:10}}>
      <div style={{display:"flex",alignItems:"center",gap:8,marginBottom:12}}>
        <span style={{fontWeight:900,fontSize:22,color:"#fff"}}>代行</span>
        <span style={{fontWeight:900,fontSize:22,color:"#f90"}}>なう</span>
        <span style={{fontSize:11,color:"#555",marginLeft:4}}>daiko-nau</span>
      </div>
      <button onClick={requestNotif} style={{width:"100%",padding:"10px 12px",background:notifStyle.bg,color:notifStyle.color,border:`1px solid ${notifStyle.border}`,borderRadius:8,fontWeight:700,fontSize:12,cursor:notifPerm==="default"? "pointer": "default",marginBottom:10,textAlign:"left"}}>
        {notifStyle.text}
    </div>
  </div>

```

```

</button>
<div style={{display:"flex",borderRadius:8,overflow:"hidden",marginBottom:10}}>
  [{"空き","対応中"].map(s=>(
    <button key={s} onClick={()=>setMyStatus(s)} style={{flex:1,padding:"11px
0",fontWeight:700,fontSize:15,border:"none",cursor:"pointer",background:myStatus===s?(s=
=="空き"?"#2ecc40":"#8b0000"):"#2a2a2a",color:myStatus===s?"#fff":"#666",transition:"all
0.2s"}}>
      {s=="空き"? 空き:" 対応中"}
    </button>
  ))}
</div>
<button onClick={()=>{setEmailSent(true);setTimeout(()=>setEmailSent(false),3000);}}
style={{width:"100%",padding:"10px
0",background:"#0d2244",color:"#6af",border:"none",borderRadius:8,fontWeight:600,fontSize
:13,cursor:"pointer",marginBottom:10}}>
  {emailSent?" 送信しました！ ":" メールで業者ステータスを送る"}
</button>
<div style={{display:"flex",borderBottom:"2px solid #1a1a1a"}}>
  {TABS.map(t=>(
    <button key={t} onClick={()=>setTab(t)} style={{flex:1,padding:"9px
0",background:"none",border:"none",color:tab===t?"#f90":"#666",fontWeight:tab===t?700:40
0,fontSize:11,cursor:"pointer",borderBottom:tab===t?"2px solid #f90":"2px solid
transparent",marginBottom:-2}}>{t}</button>
  ))}
</div>
</div>

<div style={{flex:1,overflowY:"auto"}}>

  {/* ホーム */}
  {tab==="ホーム"&&<div style={{padding:16}}>
    <div style={{color:"#f90",fontWeight:800,fontSize:18,marginBottom:14}}>業者一覧
  </div>
    <div style={{...S.card,marginBottom:14}}>
      <input value={newAgent.name}
onChange={e=>setNewAgent({...newAgent,name:e.target.value})} placeholder="業者名 (例:
代行A)" style={S.input}/>
      <input value={newAgent.phone}
onChange={e=>setNewAgent({...newAgent,phone:e.target.value})} placeholder="電話番号"
style={{...S.input,marginTop:8}}/>
      <button onClick={addAgent} style={S.outlineBtn}>+ 業者を追加</button>
    </div>
    {agents.map(a=>(
      <div key={a.id} style={{...S.card,borderLeft:`4px solid ${a.status==="空き
"?"#2ecc40":"#e74c3c"}`}}>
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"center"}}>
          <span style={{fontWeight:700,fontSize:16}}>{a.name}</span>

```

```

        <span style={{fontSize:11,padding:"3px
10px",borderRadius:20,fontWeight:700,background:a.status=== "空き
"? "#1a3a1a": "#3a1a1a",color:a.status=== "空き"? "#2ecc40": "#e74c3c"}}>{a.status}</span>
    </div>
    <div style={{color:"#777",fontSize:13,marginTop:4}}>{a.phone} · {a.area}</div>
    <div style={{color:"#f90",fontSize:13,marginTop:2}}>★ {a.rating}</div>
  </div>
  )))
</div>

{ /* 依頼 */
{tab=== "依頼" && <div style={{padding:16}}>
  <div style={{color:"#f90",fontWeight:800,fontSize:18,marginBottom:14}}>依頼一覧
</div>
  <div style={{...S.card,marginBottom:14}}>
    <input value={newReq.from}
onChange={e=>setNewReq({...newReq,from:e.target.value})} placeholder="依頼者名"
style={S.input}/>
    <input value={newReq.area}
onChange={e=>setNewReq({...newReq,area:e.target.value})} placeholder="エリア"
style={{...S.input,marginTop:8}}/>
    <input value={newReq.time}
onChange={e=>setNewReq({...newReq,time:e.target.value})} placeholder="時間 (例 : 23:00)"
style={{...S.input,marginTop:8}}/>
    <button onClick={addRequest}
style={{...S.outlineBtn,background:"#1a2a0a",borderColor:"#2ecc40",color:"#2ecc40"}}> 🚗
依頼を追加する</button>
  </div>
  {requests.map(r=>(
    <div key={r.id} style={{...S.card,borderLeft:`4px solid ${r.status=== "待機中
"? "#3498db": "#e74c3c"}`}}>
      <div style={{display:"flex",justifyContent:"space-between"}}>
        <span style={{fontWeight:700}}>{r.from} さん</span>
        <span style={{fontSize:11,padding:"3px
10px",borderRadius:20,fontWeight:700,background:r.status=== "待機中
"? "#1a2a3a": "#3a1a1a",color:r.status=== "待機中"? "#3498db": "#e74c3c"}}>{r.status}</span>
      </div>
      <div style={{color:"#777",fontSize:13,marginTop:4}}>{r.area} · {r.time}</div>
    </div>
  )))
</div>

{ /* 地図 */
{tab=== "地図" && <div style={{padding:16}}>
  <div style={{color:"#f90",fontWeight:800,fontSize:18,marginBottom:12}}> 🗺️ マップ
</div>
  <button onClick={getLocation} style={{width:"100%",padding:"12px
0",background:"#0d2244",color:"#6af",border:"1px solid

```



```

#3498db",borderRadius:10,fontWeight:700,fontSize:14,cursor:"pointer",marginBottom:8}}>
 現在地を取得する</button>
    {geoMsg&&<div
style={{fontSize:13,textAlign:"center",marginBottom:10,color:geoMsg.startsWith("✅")?"#2ecc40":geoMsg.startsWith("❌")?"#e74c3c":"#f90"}}>{geoMsg}</div>
    <LeafletMap currentPos={currentPos} fromPos={fromPos} toPos={toPos}/>
    <div style={{...S.card,marginTop:14}}>
    <div style={{color:"#f90",fontWeight:800,fontSize:15,marginBottom:14}}>  距離計算
</div>
    <div style={{display:"flex",alignItems:"center",gap:8,marginBottom:6}}>
    <div
style={{width:10,height:10,borderRadius:"50%",background:"#2ecc40",flexShrink:0}}/>
    <input value={fromInput}
onChange={e=>{setFromInput(e.target.value);setFromPos(null);setDistance(null);}}
placeholder="出発地(例:渋谷駅)" style={{...S.input,flex:1}}/>
    </div>
    <div style={{paddingLeft:4,marginBottom:6}}><div
style={{width:2,height:16;background:"#333",marginLeft:4}}/></div>
    <div style={{display:"flex",alignItems:"center",gap:8,marginBottom:14}}>
    <div
style={{width:10,height:10,borderRadius:"50%",background:"#e74c3c",flexShrink:0}}/>
    <input value={toInput}
onChange={e=>{setToInput(e.target.value);setToPos(null);setDistance(null);}} placeholder="
目的地(例:太宰府、東京タワー)" style={{...S.input,flex:1}}/>
    </div>
    <div style={{display:"flex",gap:8}}>
    <button onClick={doCalc} style={{flex:1,padding:"13px
0",background:"#f90",color:"#000",border:"none",borderRadius:10,fontWeight:800,fontSize:1
4,cursor:"pointer"}}>  距離を計算</button>
    <button onClick={openGmap} style={{flex:1,padding:"13px
0",background:"#1a6b3a",color:"#4efa8a",border:"1px solid
#2ecc70",borderRadius:10,fontWeight:800,fontSize:14,cursor:"pointer"}}>  Googleマップ
</button>
    </div>
    {calcMsg&&<div
style={{marginTop:10,fontSize:13,textAlign:"center",color:calcMsg.startsWith("✅")?"#2ecc40":calcMsg.startsWith("❌")||calcMsg.startsWith("⚠️")?"#e74c3c":"#f90"}}>{calcMsg}</div>
    {distance!==null&&(
    <div style={{marginTop:14;background:"#111",borderRadius:12,padding:"18px
14px",border:"1px solid #f90",textAlign:"center"}}>
    <div style={{color:"#555",fontSize:12,marginBottom:4}}>直線距離</div>
    <div
style={{color:"#f90",fontSize:38,fontWeight:900,lineHeight:1}}>{distance}>=1?`${distance.toFixed(1)} km`:`${(distance*1000).toFixed(0)} m`</div>
    {distance>=1&&<div style={{color:"#555",fontSize:12,marginTop:8}}>≈ 車で約
    {Math.round(distance/40*60)} 分(40km/h想定)</div>
    <div style={{color:"#444",fontSize:11,marginTop:4}}>※直線距離・実走行距離とは
    異なります</div>

```

```

    <button onClick={openGmap} style={{marginTop:12,width:"100%",padding:"12px
0",background:"#1a6b3a",color:"#4efa8a",border:"1px solid
#2ecc70",borderRadius:10,fontWeight:800,fontSize:14,cursor:"pointer"}}> 🗺 Googleマップで
ルートを開く → </button>

```

```

    </div>

```

```

  })

```

```

</div>

```

```

</div>}

```

```

{/* 業者 */}

```

```

{tab==="業者"&&<div style={{padding:16}}>

```

```

  <div style={{color:"#f90",fontWeight:800,fontSize:18,marginBottom:14}}>業者管理

```

```

</div>

```

```

  {agents.map(a=>(

```

```

    <div key={a.id} style={S.card}>

```

```

      <div style={{display:"flex",justifyContent:"space-between",alignItems:"center"}}>

```

```

        <span style={{fontWeight:700,fontSize:16}}>{a.name}</span>

```

```

        <button

```

```

onClick={()=>setAgents(agents.map(x=>x.id===a.id?{...x,status:x.status==="空き"?対応中
":"空き":x))}

```

```

      style={{background:a.status==="空き
"?"#2ecc40":"#e74c3c",color:"#fff",border:"none",borderRadius:20,padding:"4px
12px",fontSize:12,fontWeight:700,cursor:"pointer"}}>

```

```

        {a.status}

```

```

      </button>

```

```

    </div>

```

```

    <div style={{color:"#777",fontSize:13,marginTop:6}}>📞 {a.phone}</div>

```

```

    <div style={{color:"#777",fontSize:13}}>📍 {a.area}</div>

```

```

  </div>

```

```

  )))

```

```

</div>}

```

```

{/* タイムライン */}

```

```

{tab==="タイムライン"&&<div>

```

```

  {/* 投稿フォーム */}

```

```

  <div style={{background:"#000",borderBottom:"1px solid #222",padding:"12px
16px",position:"sticky",top:0,zIndex:5}}>

```

```

    <div style={{display:"flex",gap:10,alignItems:"flex-start"}}>

```

```

      <Avatar name={myName||"あ"} size={40}/>

```

```

      <div style={{flex:1}}>

```

```

        <input value={myName} onChange={e=>setMyName(e.target.value)}

```

```

placeholder="ユーザー名 (例: 田中)" maxLength={20}

```

```

        style={{width:"100%",background:"transparent",border:"none",borderBottom:"1px
solid #333",color:"#aaa",fontSize:13,padding:"4px
0",marginBottom:8,outline:"none",fontFamily:"inherit"}}/>

```

```

        <textarea value={postText}

```

```

onChange={e=>setPostText(e.target.value.slice(0,140))} placeholder="いまどうしてる?"
rows={3}

```

```

style={{width:"100%",background:"transparent",border:"none",color:"#fff",fontSize:15,resize:"
none",outline:"none",fontFamily:"inherit",lineHeight:1.5}}/>
    <div
style={{display:"flex",justifyContent:"space-between",alignItems:"center",marginTop:8}}>
      <span
style={{fontSize:12,color:140-postText.length<=20?"#e74c3c":"#555"}}>{140-postText.length}
</span>
      <button onClick={addPost} disabled={!postText.trim()}
style={{padding:"8px
20px",background:postText.trim()?"#f90":"#444",color:postText.trim()?"#000":"#777",border:"
none",borderRadius:20,fontWeight:800,fontSize:14,cursor:postText.trim()?"pointer":"default",
transition:"all 0.2s"}}>
        投稿
      </button>
    </div>
  </div>
</div>
</div>
  {/* ツイート一覧 */}
  {posts.length===0&&<div
style={{textAlign:"center",color:"#444",fontSize:14,padding:"60px 20px"}}>まだ投稿がありま
せん<br/>最初の投稿をしてみましょう！ </div>}
  {posts.map(p=>(
    <Tweet key={p.id} post={p} myName={myName||"匿名ユーザー"}
onLike={handleLike} onReply={handleReply}/>
  ))}
</div>
</div>
</div>
);
}

```